

Эскалация привилегий. Shatter attack.

Доклад

Выполнила: студентка группы НКАбд-04-24

Подхалюзина Виолетта Михайловна

Студенческий билет: 1132246761

Содержание

1 Вводная часть	1
1.1 Актуальность темы	1
1.2 Цели и задачи	2
2 Теоретические основы эскалации привилегий	2
2.1 Понятие и виды эскалации привилегий	2
2.2 Архитектура изоляции процессов и уровни привилегий	2
3 Shatter Attack: эскалация через оконную подсистему	3
3.1 Архитектура обмена сообщениями Win32 API	3
3.2 Механизм и этапы реализации атаки	3
4 Механизмы защиты и противодействия	5
4.1 Изоляция нулевого сеанса (Session 0 Isolation)	5
4.2 Изоляция привилегий пользовательского интерфейса (UIPI)	5
5 Вывод	6
Список литературы	7

1 Вводная часть

1.1 Актуальность темы

В условиях глобальной цифровизации и усложнения архитектуры вычислительных систем проблема контроля доступа и строгой изоляции процессов приобретает критическое значение. Согласно ГОСТ Р 50922-2006, защита информации подразумевает предотвращение несанкционированного доступа, изменения или уничтожения данных. Однако даже при использовании передовых средств криптографической защиты и сетевых экранов операционная система (ОС) остается фундаментальным слоем, уязвимости в котором могут свести на нет любые внешние периметры безопасности.

Одной из самых серьезных угроз внутренней безопасности ОС является эскалация (повышение) привилегий. Эта фаза является неотъемлемой частью жизненного цикла большинства сложных целенаправленных кибератак (АРТ). Получив первоначальный доступ к системе с правами обычного пользователя, злоумышленник стремится повысить свои полномочия до уровня администратора или ядра системы, чтобы закрепиться в инфраструктуре, отключить средства защиты и получить доступ к конфиденциальной информации.

Исторически одной из наиболее показательных архитектурных уязвимостей, иллюстрирующих фундаментальные проблемы в разграничении прав доступа на уровне графической под-

системы, является атака типа Shatter (Shatter attack). Несмотря на то что классические векторы данной атаки были нивелированы в современных версиях семейства ОС Windows, принципы, лежащие в ее основе, до сих пор изучаются в академических курсах по информационной безопасности как классический пример нарушения доверенной среды взаимодействия процессов. Актуальность исследования данной темы обусловлена необходимостью понимания механизмов межпроцессного взаимодействия (IPC) для разработки надежных и защищенных программных комплексов.

1.2 Цели и задачи

Цель: Целью данного доклада является комплексный анализ архитектурной уязвимости, приводящей к эскалации привилегий посредством манипуляции подсистемой оконных сообщений (Shatter attack), а также изучение современных программных механизмов защиты ОС, направленных на противодействие угрозам данного класса.

Задачи:

Дать строгое определение понятию эскалации привилегий и рассмотреть принципы разграничения доступа в операционных системах. Проанализировать архитектурные особенности графической подсистемы обмена сообщениями Win32 API и выявить коренные причины возникновения уязвимостей межпроцессного взаимодействия. Детально разобрать механизм и этапы реализации классической Shatter-атаки на процесс с повышенными привилегиями. Рассмотреть современные подходы к защите (на примере технологий Mandatory Integrity Control и Session 0 Isolation), опираясь на труды отечественных исследователей в области архитектуры ОС.

2 Теоретические основы эскалации привилегий

2.1 Понятие и виды эскалации привилегий

В современной теории информационной безопасности, как отмечает профессор МГУ В.А. Сухомлин (Сухомлин 2021 г.), безопасность вычислительной системы базируется на строгой аутентификации субъектов и авторизации их доступа к объектам. Эскалация привилегий — это процесс несанкционированного получения субъектом (пользователем или процессом) более высоких прав доступа путем эксплуатации логических ошибок, уязвимостей нулевого дня или архитектурных недочетов в программном обеспечении.

Принято классифицировать эскалацию привилегий на два основных вектора:

Вертикальная эскалация (Privilege Escalation): процесс с низкими или стандартными правами пользователя преодолевает границы безопасности и получает права суперпользователя (root в UNIX-подобных системах или NT AUTHORITY\SYSTEM в Windows). Это позволяет атакующему полностью контролировать ядро системы. **Горизонтальная эскалация:** злоумышленник получает доступ к ресурсам и процессам другого пользователя с аналогичным уровнем прав. Это часто используется для обхода системных квот или кражи сессионных токенов конкретного сотрудника.

2.2 Архитектура изоляции процессов и уровни привилегий

В учебных пособиях Д.С. Кулябова и А.В. Корольковой (Кулябов и Королькова 2018 г.), посвященных архитектуре операционных систем, неоднократно подчеркивается, что фундаментом защиты является аппаратная поддержка так называемых «колец защиты» (Protection Rings). Современные процессоры архитектуры x86/x64 обеспечивают физическое разделение адресных пространств.

Ядро операционной системы, драйверы устройств и критически важные службы исполняются в нулевом кольце (Ring 0), имея прямой доступ к аппаратному обеспечению. Пользовательские приложения запускаются в третьем кольце (Ring 3). Переход из Ring 3 в Ring 0 строго

регламентирован механизмом системных вызовов (syscalls). Любая уязвимость, позволяющая пользовательскому процессу выполнить произвольный код в контексте процесса, запущенного от имени системы, автоматически приводит к компрометации Ring 0 (рис. 1).

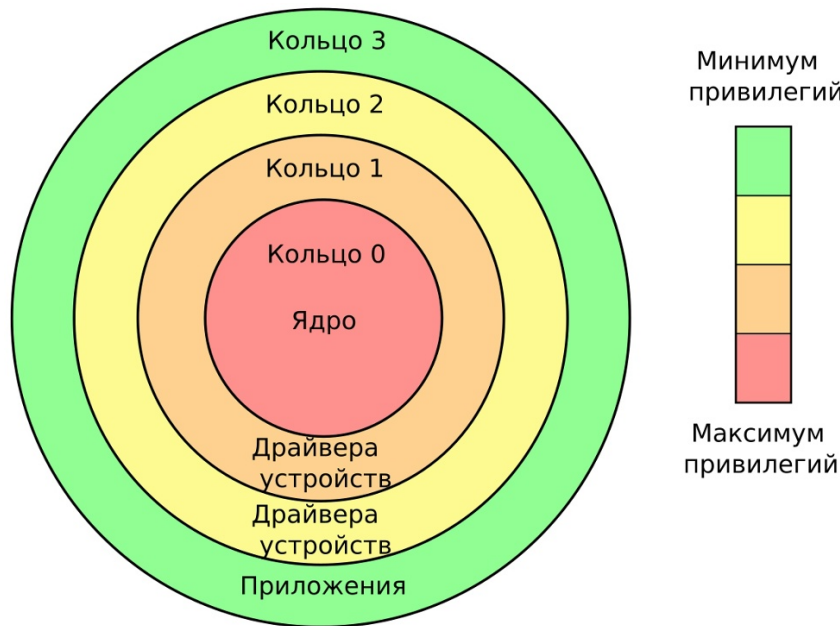


Рис. 1: Иерархия колец защиты (Protection Rings) и концепция изоляции адресных пространств

3 Shatter Attack: эскалация через оконную подсистему

3.1 Архитектура обмена сообщениями Win32 API

Чтобы понять природу Shatter-атаки, необходимо рассмотреть, как функционирует подсистема пользовательского интерфейса (User Interface, UI) в ОС Windows. В отличие от многих UNIX-систем, где графическая оболочка (например, X-сервер) исторически была отделена от ядра, в ранних версиях Windows (особенно до Windows Vista) графическая подсистема была глубоко интегрирована с системными компонентами (Руссинович и др. 2018 г.).

Каждое окно в Windows обладает собственной процедурой обработки (Window Procedure). Операционная система и приложения общаются друг с другом посредством отправки структурированных сообщений (Messages). Для этого используются системные вызовы SendMessage и PostMessage. Например, при нажатии кнопки мыши ОС генерирует сообщение WM_LBUTTONDOWN и отправляет его в очередь сообщений конкретного окна.

Фундаментальная архитектурная ошибка, заложенная на этапе проектирования Win32 API, заключалась в отсутствии проверки уровня привилегий процесса-отправителя по отношению к процессу-получателю в рамках одной интерактивной сессии (одного рабочего стола). Это означало, что любой процесс с правами обычного пользователя мог послать легитимное системное сообщение любому другому окну на том же рабочем столе, даже если это окно принадлежало процессу с правами SYSTEM (например, графическому интерфейсу антивируса).

3.2 Механизм и этапы реализации атаки

Концепция атаки "Shatter attack" (от англ. "разбивать вдребезги") была впервые детально описана исследователем безопасности Крисом Пэджетом в 2002 году. Она продемонстрировала,

как доверие к механизмам графического интерфейса можно использовать для полного обхода политик разграничения доступа.

Атака базируется на манипулировании параметрами сообщений, передаваемых целевому окну. Сообщения в Windows часто содержат два дополнительных параметра: `wParam` и `lParam`. Для некоторых системных сообщений, таких как `WM_TIMER` (таймер) или сообщений элементов управления (например, `EM_SETWORDBREAKPROC` для текстовых полей), эти параметры используются для передачи указателей на функции обратного вызова (callback functions) в памяти процесса-получателя.

Алгоритм реализации классической Shatter-атаки включает следующие шаги:

Разведка и выбор цели: Атакующий процесс сканирует рабочий стол с помощью функции `EnumWindows` или `FindWindow` в поисках окна, принадлежащего службе с системными привилегиями. Окно может быть даже скрытым (невидимым для пользователя).

Внедрение вредоносного кода (Payload Injection): Злоумышленник размещает шелл-код (небольшую вредоносную программу, например, запускающую командную строку `cmd.exe`) в адресном пространстве целевого процесса. На старых системах это можно было сделать с помощью сообщения `WM_SETTEXT`, заставив окно сохранить нужный набор байт в своей памяти.

Отправка триггерного сообщения: Атакующий использует функцию `SendMessage` для отправки специального сообщения целевому окну. В качестве параметра передается адрес ранее внедренного шелл-кода.

Перехват потока выполнения (Execution): Когда привилегированный процесс обрабатывает полученное сообщение, он интерпретирует переданный параметр как легитимный адрес функции и передает управление по этому адресу. В результате вредоносный код выполняется в контексте `NT AUTHORITY\SYSTEM` (рис. рис. 2).

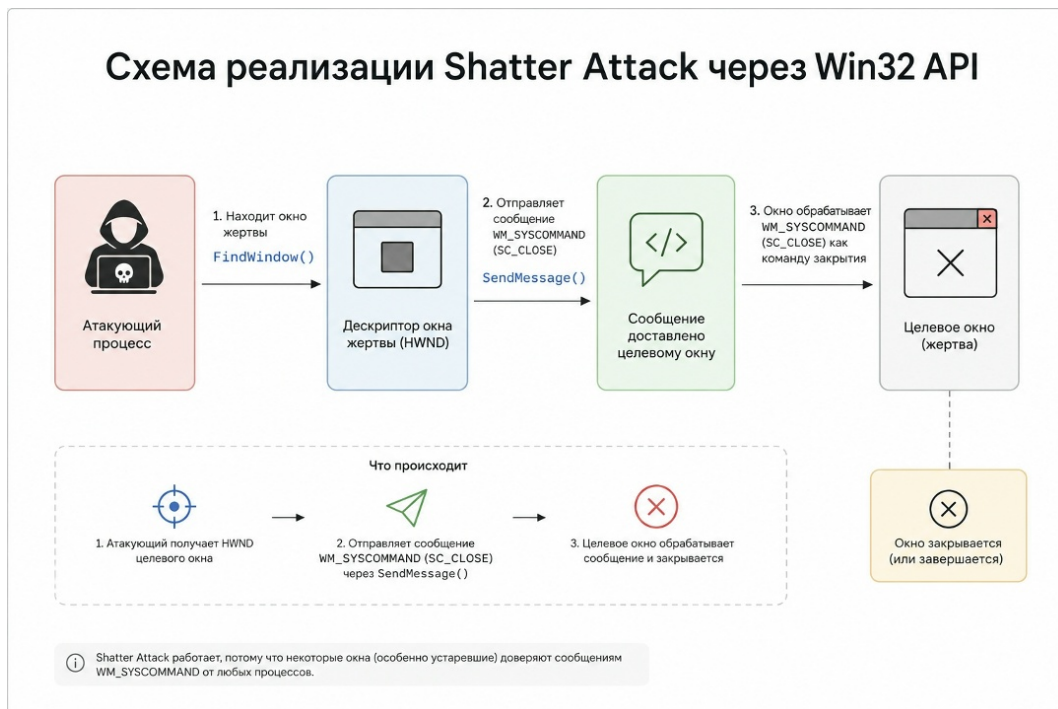


Рис. 2: Механизм передачи оконных сообщений и перехвата потока выполнения в классической Shatter-атаке.

Опасность этой уязвимости заключалась в том, что она эксплуатировала не ошибку программиста в конкретном приложении, а фундаментальный принцип работы всей графической подсистемы Windows.

4 Механизмы защиты и противодействия

Эволюция методов защиты от атак класса Shatter является отличным примером того, как разработчики ОС переходят от дискреционного (DAC) к мандатному контролю доступа (MAC). Как подчеркивается в трудах А.А. Малюка (Малюк 2020 г.) по методологическим основам защиты информации, надежность системы достигается только тогда, когда политики безопасности реализуются на уровне архитектуры без возможности их обхода через пользовательские API.

4.1 Изоляция нулевого сеанса (Session 0 Isolation)

До выхода Windows Vista первая учетная запись, вошедшая в систему, а также все системные службы разделяли общий “Сеанс 0” (Session 0). Это означало, что графические окна служб отображались на том же интерактивном рабочем столе (Interactive Desktop), что и окна пользователя. Именно это позволяло реализовать Shatter-атаку, так как процессы “видели” друг друга.

Microsoft внедрила архитектурное изменение, получившее название «Изоляция нулевого сеанса». Суть механизма заключается в том, что Сеанс 0 теперь жестко зарезервирован исключительно для неинтерактивных системных служб. Когда первый пользователь входит в систему, для него создается новый изолированный Сеанс 1. Последующие пользователи получают Сеансы 2, 3 и так далее.

Поскольку службы и пользовательские приложения теперь находятся в физически разных сеансах, они не имеют общего графического рабочего стола. Следовательно, процесс пользователя из Сеанса 1 больше не может использовать FindWindow для обнаружения окна системной службы в Сеансе 0 и не может отправить ему сообщение (рис. рис. 3).



Рис. 3: Разделение сеансов пользователя и системных служб как метод противодействия IPC-атакам.

4.2 Изоляция привилегий пользовательского интерфейса (UIPI)

Изоляции нулевого сеанса было недостаточно для защиты от эскалации привилегий в рамках одного пользовательского сеанса (например, когда пользователь запускает одно приложение

с обычными правами, а другое — от имени администратора через UAC). Для решения этой задачи была разработана технология UIPI (User Interface Privilege Isolation).

UIPI является надстройкой над механизмом обязательного контроля целостности (Mandatory Integrity Control, MIC), который концептуально схож с классической мандатной моделью Белла-ЛаПадулы, описываемой Д.С. Кулябовым (Кулябов и Королькова 2018 г.). В архитектуре MIC каждому процессу назначается уровень целостности (Integrity Level):

Low (Низкий) — для процессов с высоким риском компрометации (например, веб-браузеры). Medium (Средний) — стандартный уровень для обычных приложений. High (Высокий) — процессы, запущенные от имени администратора. System (Системный) — системные службы ядра. Технология UIPI аппаратно и программно запрещает процессам с более низким уровнем целостности отправлять сообщения окнам процессов с более высоким уровнем целостности. Любые попытки вызова SendMessage в адрес высокопривилегированного процесса просто игнорируются на уровне ядра подсистемы win32k.sys. Разрешен лишь минимальный набор безопасных сообщений (например, информирование об изменении размера экрана) (рис. рис. 4).



Рис. 4: Взаимодействие процессов на базе Mandatory Integrity Control (MIC) и фильтрация сообщений.

Таким образом, комплексное применение строгой сессионной изоляции и мандатного контроля целостности сделало реализацию классической Shatter-атаки невозможной в современных вычислительных средах.

5 Вывод

Проблема эскалации привилегий является одной из наиболее критичных в обеспечении информационной безопасности вычислительных систем (ГОСТ Р 50922-2006. Защита информации. Основные термины и определения 2007 г.). Подробный анализ атаки Shatter attack демонстрирует, что игнорирование политик разграничения доступа при разработке системных интерфейсов (на примере Win32 API и системы обмена сообщениями) может привести к возможности полного обхода механизмов безопасности даже при отсутствии ошибок переполнения буфера.

Развитие современных операционных систем показывает устойчивый тренд на усиление механизмов изоляции. Внедрение таких технологий, как изоляция сеансов (Session 0 Isolation) и мандатный контроль целостности процессов (MIC/UIPI), подтверждает тезисы отечественных ученых (В.А. Сухомлина, Д.С. Кулябова, А.А. Малюка) (Сухомлин 2021 г.; Кулябов и Королькова 2018 г.; Малюк 2020 г.) о том, что надежная защита требует комплексного архитектурного подхода. Изоляция должна применяться на всех уровнях: от файловой системы и оперативной памяти до графических интерфейсов и межпроцессного взаимодействия.

Специалистам по информационной безопасности критически важно понимать исторические векторы атак, такие как Shatter attack, поскольку принципы инъекции и манипулирования доверием между компонентами системы продолжают эволюционировать, находя отражение в современных угрозах и уязвимостях новых программных платформ (Шаньгин 2022 г.).

Список литературы

ГОСТ Р 50922-2006. *Защита информации. Основные термины и определения*. 2007 г. Стандартинформ.

Кулябов, Д. С., и А. В. Королькова. 2018 г. *Архитектура и принципы построения операционных систем: Учебное пособие*. РУДН.

Малюк, А. А. 2020 г. *Информационная безопасность: концептуальные и методологические основы защиты информации. Учебное пособие*. Горячая линия-Телеком.

Руссинович, М., Д. Соломон, и А. Ионеску. 2018 г. *Внутреннее устройство Windows (Windows Internals), 7-е издание. Часть 1*. Питер.

Сухомлин, В. А. 2021 г. *Введение в анализ информационной безопасности. Теоретико-методологические основы*. Горячая линия - Телеком.

Шаньгин, В. Ф. 2022 г. *Информационная безопасность и защита информации в компьютерных системах и сетях*. ИД ФОРУМ: НИЦ ИНФРА-М.